

Análise de Eficiência e Estabilidade em Sistemas de Distribuição de Gás Natural

Contexto

Uma empresa responsável pela distribuição de gás natural atende diversos bairros de uma cidade através de estações reguladoras e redes de tubulações subterrâneas. A administração da companhia precisa monitorar constantemente a eficiência do sistema, as perdas de pressão e os custos operacionais para evitar interrupções no abastecimento.

O objetivo é desenvolver uma ferramenta computacional capaz de analisar o desempenho da rede de distribuição em diferentes cenários operacionais, identificando regiões críticas e auxiliando no planejamento da manutenção preventiva.

Descrição do Problema

Desenvolver um programa que realize a análise operacional do sistema de distribuição de gás para **N bairros**, onde **N** será informado pelo usuário a cada execução.

O sistema deverá calcular indicadores de eficiência energética, estabilidade de pressão, perdas na rede e custos operacionais, além de gerar gráficos comparativos para facilitar a análise do sistema completo.

Entradas

O programa deverá solicitar ao usuário:

- número de bairros **N**
- nome de cada bairro
- demanda diária de gás (m³/dia)
- capacidade máxima da estação reguladora (m³)
- volume inicial disponível

(m3)
percentual de perda na tubulação
(%)
pressão média da rede (bar)
custo operacional por m3
distribuído

Processamento

Para cada bairro, o programa deverá calcular:

volume efetivamente entregue
volume perdido na tubulação
eficiência da distribuição
(%)
custo diário operacional
autonomia da estação reguladora em
dias índice de estabilidade da pressão
classificação de risco operacional

Critério de Classificação de Risco

Nível de	
Risco	Autonomia
Baixo	superior a 4 dias
Médio	entre 2 e 4 dias
Alto	inferior a 2 dias

Fórmulas Sugeridas

1. Volume perdido

$$\text{Volume Perdido} = \frac{\text{Demanda} \times \text{Perda}}{100}$$

2. Volume entregue

$$\text{Volume Entregue} = \text{Demanda} - \text{Volume Perdido}$$

3. Eficiência da distribuição

$$\text{Eficiência} = \frac{\text{Volume Entregue}}{\text{Demanda}} \times 100$$

4. Custo operacional diário

$$\text{Custo} = \text{Volume Entregue} \times \text{Custo por m}^3$$

5. Autonomia

$$\text{Autonomia} = \frac{\text{Volume Inicial} - \text{Demanda}}{\text{Demanda}}$$

6. Índice de estabilidade da pressão

$$\text{IEP} = \frac{\text{Pressão Média}}{\text{Pressão Ideal}}$$

(considerar pressão ideal = 5 bar)

Saídas Obrigatórias

○ programa deverá
apresentar:

tabela completa com os resultados por bairro

- gráfico de barras comparando:

- demanda diária
- volume entregue

gráfico de barras do custo operacional por bairro

gráfico de linha da eficiência da distribuição

gráfico de pizza mostrando os níveis de risco

identificação automática do bairro mais crítico

relatório final do desempenho geral do sistema

Desafio Adicional

O sistema deverá realizar uma simulação operacional de **10 dias consecutivos**, considerando:

variação aleatória da demanda diária entre $\pm 15\%$

variação aleatória da pressão da rede

atualização automática dos volumes
disponíveis

- identificação dos dias críticos

alerta automático quando:

a autonomia ficar abaixo de 1 dia

a eficiência cair abaixo de 75%

Ao final da simulação, o programa deverá
gerar:

gráfico de linha da evolução do volume

disponível gráfico da variação da pressão ao longo

dos dias gráfico da eficiência média diária do sistema

Área

Engenharia Civil
Engenharia Sanitária

Engenharia
Mecânica
Engenharia de Energia

Tecnologias Recomendadas

Python NumPy
Pandas
Matplotlib
Random

Objetivo Acadêmico

O projeto permite aplicar conceitos de:

análise de sistemas
simulação
computacional
eficiência operacional
visualização de dados
modelagem
matemática
programação em
engenharia
tomada de decisão baseada em indicadores

```
=====
SISTEMA DE ANÁLISE DE EFICIÊNCIA E ESTABILIDADE
EM SISTEMAS DE DISTRIBUIÇÃO DE GÁS NATURAL
=====
```

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
```

```

import random
import os

=====

CONFIGURAÇÃO INICIAL
=====

PRESSAO_IDEAL = 5

if not os.path.exists("graficos"):
    os.makedirs("graficos")

=====

ENTRADA DE DADOS
=====

N = int(input("Digite o número de bairros: "))

dados = []

for i in range(N):

    print(f"\n===== Bairro {i+1} =====")

    nome = input("Nome do bairro: ")

    demanda = float(input("Demanda diária de gás (m³/dia): "))

    capacidade = float(input("Capacidade máxima da estação (m³): "))

    volume_inicial = float(input("Volume inicial disponível (m³): "))

    perda = float(input("Percentual de perda na tubulação (%): "))

    pressao = float(input("Pressão média da rede (bar): "))

    custo_m3 = float(input("Custo operacional por m³: "))

    # =====

    # PROCESSAMENTOS

    # =====

    volume_perdido = demanda * (perda / 100)

```

```
volume_entregue = demanda - volume_perdido

eficiencia = (volume_entregue / demanda) * 100

custo_operacional = volume_entregue * custo_m3

autonomia = volume_inicial / demanda

iep = pressao / PRESSAO_IDEAL

# =====

# CLASSIFICAÇÃO DE RISCO

# =====

if autonomia > 4:

    risco = "Baixo"

elif autonomia >= 2:

    risco = "Médio"

else:

    risco = "Alto"

# =====

# ARMAZENAMENTO

# =====

dados.append([

    nome,

    demanda,

    capacidade,

    volume_inicial,

    perda,
```

```
    pressao,

    volume_perdido,

    volume_entregue,

    eficiencia,

    custo_operacional,

    autonomia,

    iep,

    risco

])
```

=====

DATAFRAME

=====

```
colunas = [
    "Bairro",
    "Demanda",
    "Capacidade",
    "Volume Inicial",
    "Perda (%)",
    "Pressão",
    "Volume Perdido",
    "Volume Entregue",
    "Eficiência (%)",
    "Custo Operacional",
    "Autonomia",
    "IEP",
    "Risco"
]

df = pd.DataFrame(dados, columns=colunas)
```

=====

RESULTADOS


```
=====
print("\n===== RESULTADOS =====\n")
print(df)
```

```
=====
IDENTIFICAÇÃO DO BAIRRO MAIS CRÍTICO
=====
```

```
bairro_critico = df.loc[df["Autonomia"].idxmin()]

print("\n===== BAIRRO MAIS CRÍTICO =====")
print(bairro_critico)
```

```
=====
RELATÓRIO FINAL
=====
```

```
print("\n===== RELATÓRIO GERAL =====")

print(f"Eficiência média: {df['Eficiência (%)'].mean():.2f}%")

print(f"Custo operacional total: R$ {df['Custo Operacional'].sum():.2f}")

print(f"Autonomia média: {df['Autonomia'].mean():.2f} dias")
```

```
=====
EXPORTAÇÃO CSV
=====
```

```
df.to_csv("relatorio_final.csv", index=False)
```

```
=====
GRÁFICO 1 - DEMANDA x ENTREGA
=====
```

```
plt.figure(figsize=(10,6))

x = np.arange(len(df))

largura = 0.35

plt.bar(x - largura/2, df["Demanda"], largura, label="Demanda")

plt.bar(x + largura/2, df["Volume Entregue"], largura, label="Entregue")
```

```
plt.xticks(x, df["Bairro"])

plt.ylabel("Volume (m³)")

plt.title("Demanda x Volume Entregue")

plt.legend()

plt.savefig("graficos/demanda_vs_entrega.png")
```

GRÁFICO 2 - CUSTO OPERACIONAL

```
plt.figure(figsize=(10,6))

plt.bar(df["Bairro"], df["Custo Operacional"])

plt.ylabel("Custo")

plt.title("Custo Operacional por Bairro")

plt.savefig("graficos/custo_operacional.png")
```

GRÁFICO 3 - EFICIÊNCIA

```
plt.figure(figsize=(10,6))

plt.plot(df["Bairro"], df["Eficiência (%)"], marker='o')

plt.ylabel("Eficiência (%)")

plt.title("Eficiência da Distribuição")

plt.grid()

plt.savefig("graficos/eficiencia.png")
```

GRÁFICO 4 - RISCO

```
riscos = df["Risco"].value_counts()
```

```

plt.figure(figsize=(8,8))

plt.pie(
riscos,
labels=riscos.index,
autopct='%1.1f%%'
)

plt.title("Níveis de Risco")

plt.savefig("graficos/riscos.png")

=====

SIMULAÇÃO DE 10 DIAS

=====

print("\n===== SIMULAÇÃO OPERACIONAL =====")

simulacao = []

volume_disponivel = df["Volume Inicial"].sum()

for dia in range(1, 11):

    demanda_total = 0

    eficiencia_media = 0

    pressao_media = 0

    for i in range(len(df)):

        variacao_demanda = random.uniform(-0.15, 0.15)

        demanda_dia = df.loc[i, "Demanda"] * (1 + variacao_demanda)

        variacao_pressao = random.uniform(-0.10, 0.10)

        pressao_dia = df.loc[i, "Pressão"] * (1 + variacao_pressao)

        perda = df.loc[i, "Perda (%)"]

        volume_perdido = demanda_dia * (perda / 100)

        volume_entregue = demanda_dia - volume_perdido

```

```

    eficiencia = (volume_entregue / demanda_dia) * 100

    demanda_total += demanda_dia

    eficiencia_media += eficiencia

    pressao_media += pressao_dia

eficiencia_media /= len(df)

pressao_media /= len(df)

volume_disponivel -= demanda_total

autonomia = volume_disponivel / demanda_total

# ALERTAS

if autonomia < 1:

    print(f"ALERTA: autonomia crítica no dia {dia}")

if eficiencia_media < 75:

    print(f"ALERTA: eficiência baixa no dia {dia}")

simulacao.append([

    dia,

    demanda_total,

    volume_disponivel,

    eficiencia_media,

    pressao_media,

    autonomia

])

```

```

=====
DATAFRAME DA SIMULAÇÃO
=====

```

```

df_sim = pd.DataFrame(simulacao, columns=[
    "Dia",
    "Demanda Total",
    "Volume Disponível",
    "Eficiência Média",
    "Pressão Média",
    "Autonomia"
])

print("\n===== RESULTADOS DA SIMULAÇÃO =====\n")

print(df_sim)

=====
EXPORTAÇÃO CSV DA SIMULAÇÃO
=====

df_sim.to_csv("simulacao_10_dias.csv", index=False)

=====
GRÁFICO SIMULAÇÃO - VOLUME
=====

plt.figure(figsize=(10,6))

plt.plot(df_sim["Dia"], df_sim["Volume Disponível"], marker='o')

plt.title("Evolução do Volume Disponível")

plt.xlabel("Dias")

plt.ylabel("Volume")

plt.grid()

plt.savefig("graficos/volume_simulacao.png")

=====
GRÁFICO SIMULAÇÃO - PRESSÃO
=====

plt.figure(figsize=(10,6))

plt.plot(df_sim["Dia"], df_sim["Pressão Média"], marker='o')

```

```

plt.title("Variação da Pressão")

plt.xlabel("Dias")

plt.ylabel("Pressão")

plt.grid()

plt.savefig("graficos/pressao_simulacao.png")

=====

GRÁFICO SIMULAÇÃO - EFICIÊNCIA

=====

plt.figure(figsize=(10,6))

plt.plot(df_sim["Dia"], df_sim["Eficiência Média"], marker='o')

plt.title("Eficiência Média Diária")

plt.xlabel("Dias")

plt.ylabel("Eficiência (%)")

plt.grid()

plt.savefig("graficos/eficiencia_simulacao.png")

=====

FINALIZAÇÃO

=====

print("\nSistema finalizado com sucesso!")

print("Arquivos CSV e gráficos foram gerados.")

```

Show less

34 minutes ago

Planning web app data analysis features (10 seconds)

This is a natural gas distribution analysis system. I'll build it as an interactive web dashboard where you can enter neighborhood data, view analysis results with charts, and run the 10-day simulation — all in the browser, no Python required.

Let me create the app now.

3 actions

Now let me read the key reference files and the scaffolded files simultaneously.

4 actions

Now let me read the CSS overrides reference and then build the full app.

2 actions

Now I'll build all the files in parallel — types, calculations, pages, and the updated CSS and App entry point.

4 actions

Let me write all the remaining files in parallel.

5 actions

Now let me fix the CSS (replace all the `red` placeholder values with real colors) and start the workflow in parallel.

3 actions

Let me check the logs and take a screenshot to verify the app is running correctly.

2 actions

There's a duplicate `riscoColor` declaration in `Analysis.tsx` — I'll fix it now.

2 actions

I need to add the `getBadgeClass` helper function that I referenced. Let me also check what else needs fixing.

6 actions

The log is still showing the old cached error. Let me read the current file to see if there's still a `riscoColor` function lingering inside the component.

2 actions

Let me read the end of `Analysis.tsx` to find the remaining duplicate.

4 actions

Found it — there's still a `riscoColor` arrow function inside the component at line 73. Let me remove it.

6 actions

The HMR updated cleanly with no errors. Let me take a screenshot to confirm the app looks good.

2 actions

The app is live and working beautifully. Let me present it now.

Presented output

Análise de Distribuição de Gás NaturalData Visualization

Open

Summarizing current app state

Your gas analysis system is now a fully interactive web app. Here's what it does:

Data Entry screen — fill in each neighborhood's name, daily demand, capacity, initial volume, pipeline loss percentage, average pressure, and cost per m³. Add as many neighborhoods as you like, and remove any with a click.

Analysis screen — once you click "Analisar", you get:

- KPI cards (average efficiency, total cost, average autonomy, most critical neighborhood)
- Bar chart of demand vs. volume delivered
- Pie chart of risk distribution (Low / Medium / High)
- Line chart of efficiency per neighborhood
- Bar chart of operational cost per neighborhood
- Full results table with color-coded efficiency, autonomy, and risk badges
- CSV export button

Simulation screen — click "Simulação 10 Dias" to run the 10-day operational simulation with $\pm 15\%$ demand variance and $\pm 10\%$ pressure variance. You get area/line charts for volume evolution, pressure variation, average daily efficiency, and total demand — plus automatic alerts when autonomy drops below 1 day or efficiency falls below 75%. All data is also exportable as CSV.

The risk classification logic is identical to the original Python script (autonomia > 4 = Low, ≥ 2 = Medium, < 2 = High), and so are all formulas for efficiency, IEP, operational cost, and pipeline losses.

Next, I can add a multi-scenario comparison mode where you can save different sets of neighborhood inputs and compare their analysis results side by side. Would you like me to do that?

